

LAB 1: File and Directory Management in Linux

Author: Dr. Sandeep Kumar Sharma

Learning Objectives

The objective of this lab is to build a strong foundation in Linux file and directory management by understanding **every commonly used command**, its **important flags**, and **real-world usage patterns**. This lab focuses on *how* and *why* each command is used, not just *what* it does.

Learning Outcomes

After completing this lab, learners will be able to:

- Navigate the Linux file system confidently
 - Create single and multiple directories using different techniques
 - Create, view, copy, move, rename, and delete files safely
 - Understand the behavior of each command with its flags
 - Apply file management commands in real production and server scenarios
-

Step 1: Understanding Your Current Location – `pwd`

Command: `pwd` (Print Working Directory)

Example:

```
pwd
```

Explanation: This command shows the **absolute path** of your current directory. Before performing any file operation, it is always a best practice to confirm where you are working, especially on production servers.

Step 2: Moving Inside the File System – `cd`

Command: `cd` (Change Directory)

Common Usage:

```
cd /home  
cd /etc
```

Important Variations:

- `cd ..` → Move one directory up
- `cd .` → Stay in the same directory
- `cd ~` → Move to the current user's home directory
- `cd -` → Switch back to the previous directory

Explanation: The `cd` command changes your working location. Linux always performs operations relative to the current directory unless an absolute path is provided.

Step 3: Listing Files and Directories – `ls` (All Important Flags)

Command: `ls`

Basic Usage:

```
ls
```

Important Flags:

- `ls -l` → Long listing (permissions, owner, group, size, date)
- `ls -a` → Show hidden files (files starting with `.`)
- `ls -h` → Human-readable file sizes
- `ls -R` → Recursive listing
- `ls -t` → Sort by modification time
- `ls -r` → Reverse order
- `ls -S` → Sort by file size
- `ls -i` → Show inode numbers

Combined Examples

```
ls -lah  
ls -ltr
```

Explanation: The `ls` command is used constantly by Linux administrators to inspect directories, debug issues, and verify file presence.

Step 4: Creating Directories - `mkdir`

Command: `mkdir`

Single Directory:

```
mkdir project
```

Multiple Directories at Once:

```
mkdir dev test prod
```

Directory Hierarchy Creation:

```
mkdir -p app/config/logs
```

Important Flags:

- `-p` → Create parent directories if they do not exist
- `-v` → Verbose output
- `-m` → Set permissions during creation

Examples:

```
mkdir -v data  
mkdir -m 755 public_dir  
mkdir -vm 700 secure_dir
```

Explanation: `mkdir` is heavily used in automation, scripting, and server provisioning. The `-p` flag is extremely important in real-world scenarios.

Step 5: Creating Files - `touch`

Command: `touch`

Create Single File:

```
touch file1.txt
```

Create Multiple Files:

```
touch file1 file2 file3
```

Set Custom Timestamp:

```
touch -t 202601011200 report.txt
```

Explanation: The `touch` command creates empty files or updates timestamps. It is commonly used to prepare configuration files.

Step 6: Viewing File Content – `cat`, `less`, `more`, `head`, `tail`

`cat`

```
cat file.txt  
cat -n file.txt
```

`less` and `more`

```
less largefile.log  
more file.txt
```

`head` and `tail`

```
head -n 5 file.txt  
tail -n 5 file.txt  
tail -f /var/log/messages
```

Explanation: These commands are essential for reading configuration files and monitoring logs in real-time environments.

Step 7: Copying Files and Directories – `cp`

Command: `cp`

Basic Copy:

```
cp file1 file2
```

Copy Directory:

```
cp -r dir1 dir2
```

Important Flags:

- `-r` → Recursive copy
- `-i` → Interactive confirmation
- `-v` → Verbose output
- `-p` → Preserve permissions

Example:

```
cp -rv source backup
```

Explanation: The `cp` command duplicates data. Recursive copy is mandatory for directories.

Step 8: Moving and Renaming - `mv`

Command: `mv`

Rename File:

```
mv oldname.txt newname.txt
```

Move File:

```
mv file.txt /tmp/
```

Important Flags:

- `-i` → Ask before overwrite
- `-v` → Show progress

Explanation: `mv` performs both renaming and moving operations, making it a powerful single command.

Step 9: Deleting Files and Directories – `rm` (Critical Command)

Command: `rm`

Delete File:

```
rm file.txt
```

Delete Directory:

```
rm -r dir
```

Important Flags:

- `-i` → Ask before deletion
- `-r` → Recursive deletion
- `-f` → Force deletion

Dangerous Example (Use Carefully):

```
rm -rf dir
```

Explanation: Linux does not have a recycle bin. Deleted files are permanently removed.

Step 10: Searching Files – `find` and `locate`

`find`

```
find / -name file.txt  
find . -type f  
find / -size +10M
```

`locate`

```
locate filename
```

Explanation: `find` searches in real time, while `locate` uses an indexed database and is much faster.

Step 11: File Information and Metadata – `stat` and `file`

`stat`

```
stat file.txt
```

`file`

```
file file.txt
```

Explanation: These commands provide detailed metadata such as inode number, timestamps, and file type.

Step 12: Creating Bulk Directories and files

This step is extremely important from an **administrator, DevOps, and interview perspective**. In real environments, tasks are rarely done one-by-one.

A. Create 100 Directories at Once (Multiple Methods)

Method 1: Using Brace Expansion (Recommended)

```
mkdir dir{1..100}
```

Explanation: The shell expands `{1..100}` before executing the command. This is the fastest and cleanest method used in real-world Linux administration.

Example Verification:

```
ls dir* | wc -l
```

Method 2: Create 100 Directories with Meaningful Names

```
mkdir student{1..100}
```

Creates directories:

```
student1 student2 student3 ... student100
```

Method 3: Using Loop (Automation Friendly)

```
for i in {1..100}
do
    mkdir batch_dir$i
done
```

Explanation: Loops are commonly used in shell scripts and automation pipelines.

B. Create 100 Files at Once

Using Brace Expansion:

```
touch file{1..100}.txt
```

Verification:

```
ls file*.txt | wc -l
```

C. Create 100 Directories with Subdirectories

```
mkdir -p user{1..100}/data
```

Explanation: This command creates parent and child directories together. Very common in enterprise setups for user or application data segregation.
